



Serviços de mensageria

Tecnologias relacionadas à comunicação de sistemas para fins de integração, com ênfase em sistemas de mensageria.

Prof. Ricardo Sant'Ana

Propósito

O conhecimento de tecnologias relacionadas à integração de sistemas, especificamente, a comunicação entre sistemas por serviços de mensageria, é fundamental para essa nova realidade na qual mais e mais sistemas de software são desenvolvidos e precisam ser integrados para trazer mais valor para o usuário.

Preparação

É necessário instalar o Python 3.6 ou superior, ambiente de desenvolvimento Java versão 18 ou superior, e uma máquina (virtual ou física) Linux para instalação do RabbitMQ.

Objetivos

- Reconhecer as vantagens e desvantagens de um sistema de mensageria.
- Aplicar a tecnologia JMS para integração de sistemas desenvolvidos em Java por meio de serviços de mensageria.
- Identificar a vantagem de utilização do servidor de mensagerias RabbitMQ como solução na integração de sistemas.
- Reconhecer as principais aplicações do GraphQL na integração de sistemas.

Introdução

É notória a crescente necessidade por integração de sistemas das mais diversas tecnologias. No passado, diversos sistemas foram desenvolvidos e agora, com o intuito de trazer maior valor para o cliente ou até mesmo para a organização, há a necessidade de que sistemas conversem com sistemas. Portanto, este conteúdo é fundamental para a compreensão dos principais desafios na área de integração de sistemas.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Introdução e tipos de comunicação entre sistemas

Comunicação de sistemas

Confira os detalhes da comunicação de sistemas!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Uma das motivações para se aprofundar nos conhecimentos da comunicação entre sistemas é a crescente utilização da arquitetura de microsserviços.

Mas o que são **microsserviços**? Conforme a arquitetura dos sistemas evolui e mais complexidade é adicionada aos sistemas, muitas vezes faz sentido separar responsabilidades entre vários serviços, chamados de microsserviços.



Saiba mais

Uma arquitetura de microsserviço pode ser definida como um padrão de arquitetura que organiza um aplicativo como uma coleção de serviços de baixa granularidade e fracamente acoplados, comunicando-se por meio de protocolos “leves”.

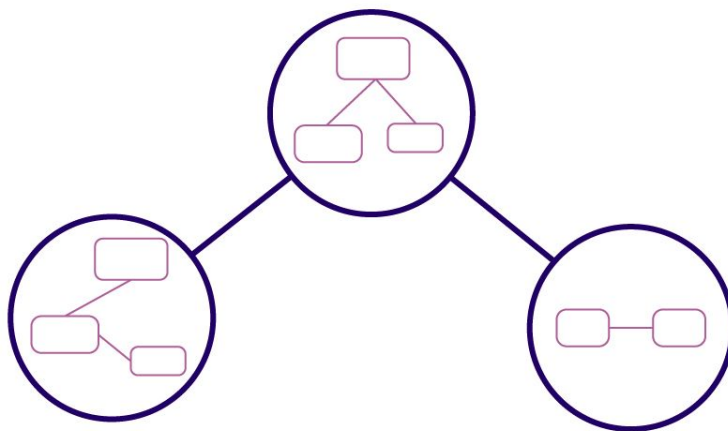
Ao dizer que é um **padrão de arquitetura**, estamos dizendo que é uma forma de organizar algo. Nesse caso, o algo é um aplicativo. E essa forma de organizar é como coleção de serviços de baixa granularidade e fracamente acoplados. Ou seja, cada serviço será responsável por uma funcionalidade preferencialmente simples (baixa granularidade) e independente de outras (fracamente acoplados).



Comentário

Um dos objetivos dos microsserviços é que as equipes possam desenvolver e implantar seus serviços independentemente de outras equipes. Isso é alcançado pela redução de várias dependências na base de código, permitindo que os desenvolvedores evoluam seus serviços de forma independente.

Nessa separação, é comum que os novos serviços sejam organizados por domínios, de forma que cada um tenha apenas uma responsabilidade e que consiga ser o mais independente possível. Por esse motivo, os contextos delimitados costumam ser ótimos candidatos a se tornarem esses novos serviços, veja:



Visão de domínios para microserviços.

Mas você pode estar se perguntando: **como**? Simples, microserviços geralmente se comunicam por mensageria!

Tipos de comunicação

Confira os tipos de comunicação de sistemas!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

A utilização de microserviços tem se tornado mais comum, e, ao adotar essa estratégia, o primeiro grande desafio que aparece é responder a seguinte pergunta:

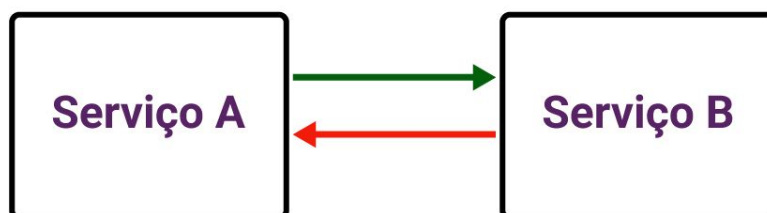
Como lidar com a comunicação entre serviços?

O que antes era uma simples chamada de métodos de outras classes utilizando uma linguagem de programação orientada a objetos, agora passa a ser uma comunicação entre sistemas que podem, potencialmente, ser implementados com tecnologias bem diferentes!

Comunicação síncrona

O jeito mais simples e comum de resolver esse desafio é usando uma **abordagem síncrona**. Nesse caso, o **serviço A** faz uma requisição para o **serviço B**, que responde imediatamente à requisição, indicando **sucesso** ou **falha**.

Esse cenário é muito similar a uma simples chamada de métodos, mas adiciona uma complexidade extra, porque a requisição e a resposta são trafegadas pela rede, veja:



Requisição síncrona.

Podemos observar que a comunicação síncrona entre serviços é simples, mas possui algumas **desvantagens**:

- Aquele que faz a requisição precisa, no mínimo, conhecer bem o serviço que está chamando, como qual é o endereço do serviço, os seus parâmetros e seus formatos.
- O requisitante do serviço também precisa conseguir lidar com qualquer tipo de erro que possa acontecer durante a requisição, como respostas erradas ou inesperadas, serviço lento, serviço fora do ar.
- É preciso saber lidar com problemas de desempenho: quando a requisição do serviço A para o serviço B depende de o B fazer outras requisições para outros serviços, que fazem outras requisições também, a resposta do serviço A só virá depois que todas as outras requisições se completarem, podendo demorar consideravelmente.
- Qualquer erro em qualquer chamada intermediária de requisições pode desencadear um erro no sistema como um todo.

Comunicação assíncrona

Uma forma de lidar com as desvantagens da comunicação síncrona entre sistemas é se comunicando assincronamente, em que as respostas **não são imediatas**.

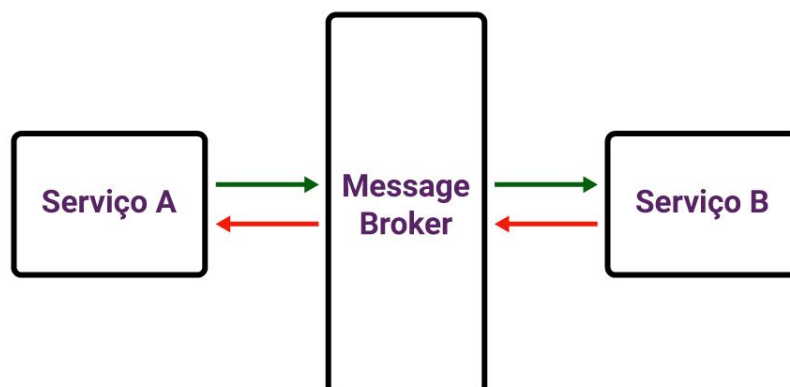
Nesse tipo de comunicação, o serviço faz a requisição da mesma forma que na comunicação síncrona, mas com uma grande diferença: o requisitante não espera até que o serviço receba a resposta. Ele pode prosseguir com o processamento de outras informações. Do mesmo modo, a resposta da requisição assíncrona deve ser enviada de forma assíncrona.



Atenção

Uma das formas de se enviar uma requisição assíncrona e receber a resposta é por meio de mensagens.

Nesse caso, cada requisição (ou mensagem) é enviada por meio de um sistema intermediário, chamado de **message broker** ou **message bus**, que entrega a mensagem ao(s) sistema(s) interessado(s), veja:



Comunicação assíncrona com um message broker.

Nesse exemplo, o serviço A envia uma mensagem ao serviço B por meio do message broker realizando uma solicitação. Nesse momento, A sabe que sua requisição foi enviada e, portanto, pode tomar providências.

Uma possível providência seria avisar ao usuário que a requisição está em processamento, mensagem comum em diversas aplicações atualmente, principalmente se você já realizou pagamentos via cartão de crédito. Observe as seguintes etapas:

- Assim que o message broker recebe a requisição de A, ele pode repassá-la para o serviço B. Após receber requisição do message broker, o serviço B sabe que deve começar a processar tal requisição.
- A resposta desse processamento será encapsulada em uma resposta a ser enviada para o serviço A, então B envia uma mensagem para A por meio do message broker.
- O message broker recebe a resposta de B e envia essa resposta para A. Quando o serviço A receber essa resposta, ele pode, por exemplo, atualizar o usuário sobre a situação da requisição.

Observe que tanto a requisição oriunda do serviço A como a resposta oriunda do serviço B são mensagens.

A comunicação assíncrona apresenta as seguintes **desvantagens**:

- Complexidade para implementar o message broker.
- Inconsistência eventual: existe um período de tempo em que um mesmo dado que exista em mais de um sistema estará atualizado em um e ainda não atualizado no outro.

Mensageria

Confira alguns detalhes e entenda o que é mensageria!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

O que é mensageria?

Podemos definir mensageria como uma abordagem de **desenvolvimento de software** que utiliza mensagens como forma de comunicação e integração (síncrona ou assíncrona) entre aplicações, usando como middleware (que fica no meio!) um message broker.

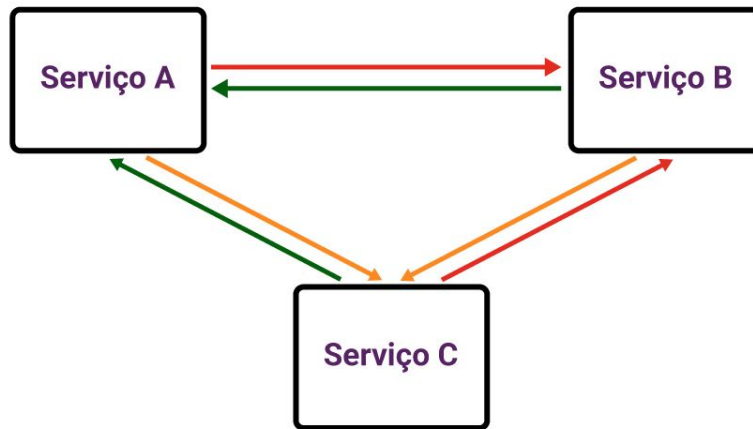
Mas afinal, o que é um message broker? É simplesmente um servidor (infraestrutura) que serve para processar e suportar o envio, recebimento, redirecionamento e monitoria das mensagens trocadas pelos diversos sistemas que passam por ele.



Curiosidade

Por se comunicarem apenas por meio do message broker, os serviços A e B precisam apenas saber o formato das mensagens recebidas e enviadas para o message broker, ou seja, os serviços que se comunicam não precisam se conhecer, trazendo um desacoplamento real que permite maior independência entre eles.

E por que ele é importante? Bom, imagine a comunicação entre 3 serviços: A, B e C. Imagine ainda que cada serviço se comunique com todos os outros (isso é apenas um exemplo!), conforme ilustrado na imagem, em que a seta representa a comunicação entre o serviço e a cor, a interface. Veja:



Exemplo de 3 serviços que se comunicam entre si.

Assim, para se acionar o serviço A, o serviço C faz uma chamada (seta verde de C para A). Para acionar o mesmo serviço A, o serviço B faz uma chamada (seta verde de B para A).



Exemplo

Imagine que o serviço A precise mudar. Nesse momento, a implementação de B e C também deverão mudar, pois a chamada ao serviço A mudou. Imagine a mesma situação para 50 serviços que utilizem o serviço A.

Se você, ao contrário de fazer a comunicação direta entre os serviços, fizer por meio de um message broker, o serviço A apenas receberá solicitações do message broker e todos os outros serviços farão solicitações para o message broker caso queiram acionar o serviço A.

Logo, uma mudança na interface do serviço A pode não afetar os outros serviços, pois o message broker faria a adaptação.

Protocolos para mensageria

Veja alguns dos principais protocolos de mensageria disponíveis no mercado. A lista não é completa, mas dará a você uma boa noção das opções:

Java Message Service (JMS)

API (Application Programming Interface) da linguagem Java para middleware orientado a mensagens. Muito usado em Servidores de Aplicação onde aplicações corporativas desenvolvidas em Java são implantadas. API é um software intermediário que permite que aplicações se comuniquem sem a necessidade de conhecer a implementação.

Advanced Message Queuing Protocol (AMQP)

Protocolo comumente utilizado em mensagens cliente/servidor e no gerenciamento de dispositivos IoT. Padrão de código aberto para mensagens assíncronas que permite mensagens criptografadas e interoperáveis entre organizações e aplicativos.

Message Queue Telemetry Transport (MQTT)

Protocolo “leve” de mensagens para uso em casos em que os clientes precisam de um pequeno espaço de código e estão conectados a redes não confiáveis ou redes com recursos de banda limitados, como IoTs. Com esse protocolo, fica mais fácil a implementação de objetos para se comunicarem em rede e enviarem mensagens, seja entre objetos ou mesmo para outros sistemas que precisam de notificações desses dispositivos.

Simple/Streaming Text Oriented Messaging Protocol (STOMP)

Protocolo que tem como foco o streaming de dados, os quais são serializados e a informação é entregue praticamente em tempo real para os consumidores.

Microsoft Message Queuing (MSMQ)

Protocolo que é uma implementação Microsoft, bastante usado em aplicações desenvolvidas em linguagem proprietária da empresa. Seu uso é bastante restrito à plataforma Microsoft.

Principais brokers de mercado

A seguir, vamos apresentar as principais implementações de brokers do mercado e relacionar com o protocolo por ele utilizado:

1

IBM MQ Series

Broker da IBM que suporta apenas o protocolo JMS.

2

ActiveMQ

Suporta tanto o JMS quanto o AMQP, ou seja, pode ser usado tanto por aplicações Java quanto por aplicações que são mais atuais e usam protocolo multilinguagem.

3

RabbitMQ

Comumente utilizado em projetos que implementam o protocolo AMQP. Mais famoso por ser de fácil uso e bom desempenho.

4

Kafka

Utilizado em projetos de alto desempenho por meio do protocolo STOMP.

Modelos de integração por mensageria

Confira como fazer a integração de sistemas utilizando mensageria!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Modelo ponto a ponto

Agora veremos as arquiteturas mais comuns que utilizam mensageria como base para sua implementação, iniciando pelo ponto a ponto. Cada arquitetura é chamada de modelo – mas no Java é conhecida como domínio.

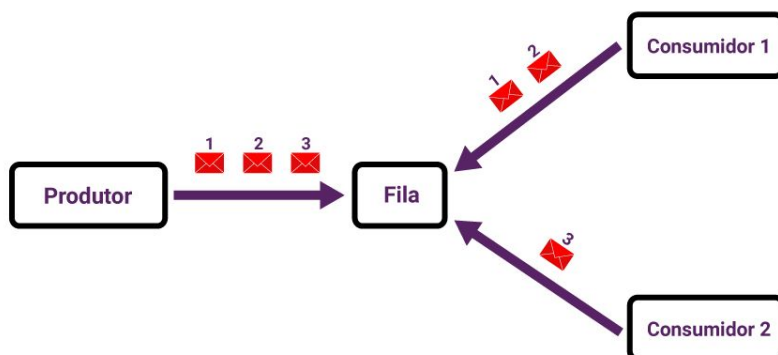


Atenção

Nesse modelo, a troca de informações entre os serviços é baseada em filas: a mensagem é enviada por uma aplicação, denominada produtor, e consumida por uma ou mais aplicações, denominadas consumidores, que “ficam ouvindo” (listening) a fila!

No processamento na fila é feita uma mensagem por vez, ou seja, somente uma mensagem por consumidor (one-to-one). Dessa forma, ainda que existam vários consumidores escutando uma mesma fila, cada consumidor recebe uma **única mensagem** por vez.

A imagem a seguir apresenta o modelo ponto a ponto. O produtor enviou a mensagem 1 e 2 que se destinam ao consumidor 1. Da mesma forma, o produtor enviou a mensagem 3 que se destina ao consumidor 2. Cada mensagem tem um único produtor e um único destino (one-to-one).



Representação do modelo ponto a ponto.

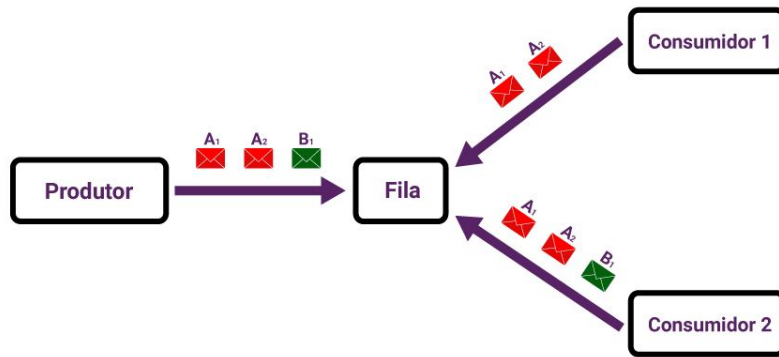
Modelo publish/subscribe

Nesse modelo, as mensagens são enviadas pela aplicação produtora e entregues para quaisquer aplicações consumidoras que assinaram o tópico da mensagem.

Vamos supor que o produtor produza, ou publica (*publish*), mensagens relacionadas a “registros de eventos de falha de autenticação” (tópico → assunto). Assim, se o consumidor 1 deseja receber mensagens desse tópico, basta ele se inscrever (assinar) na fila desse tipo de mensagem.

Esse modelo permite que uma mensagem seja entregue para vários consumidores (one-to-many), ou seja, todos os consumidores que assinaram determinado tópico receberão as mensagens produzidas.

Veja agora o modelo publish/subscribe. O consumidor 1 e 2 assinaram mensagens do tópico A (vermelho). Da mesma forma, o consumidor 2 assinou também mensagens do tópico B (amarelo). Observe que ambos os consumidores recebem as mensagens do tópico A.



Representação do modelo publish/subscribe.

Dead letter queue (DLQ)

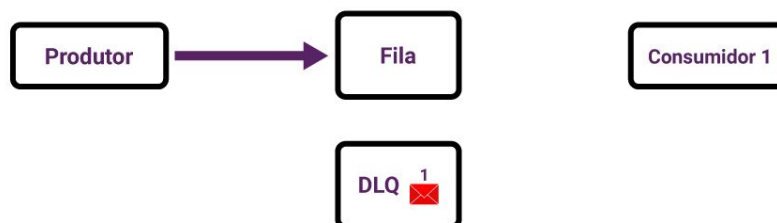
Na dead letter queue (DLQ), quando não é possível entregar uma mensagem, teremos uma fila na qual as mensagens não entregues serão adicionadas de forma que alguém tome uma ação, ou seja, não é um modelo independente, mas uma abordagem para lidar com mensagens não entregues ou não consumidas. Assim, a DLQ é responsável por receber mensagens não entregues ou não consumidas.



Atenção

Usar a DLQ para enviar mensagens de erro de integração é equivocado.

Observe um exemplo de uma mensagem que não foi consumida por ninguém e, portanto, foi enviada para DLQ para providências:



Exemplo da Dead letter queue.

Mensageria em microsserviços

A comunicação por mensageria está muito presente em projetos relacionados à arquitetura de microsserviços. Cada microsserviço deve ser independente um do outro, e ainda assim precisa compartilhar ou enviar dados para outro serviço. Ao utilizar mensageria, permitimos a comunicação com baixo acoplamento entre os serviços, o que é desejável.

Verificando o aprendizado

Questão 1

Com relação aos modelos ou domínios de fila utilizadas em serviços de mensageria, assinale a alternativa correta.

A

No modelo ponto a ponto, para que uma informação seja enviada a dois consumidores, é preciso enviar a mensagem duas vezes, uma para cada consumidor.

B

No modelo ponto a ponto é possível enviar uma mensagem para mais de um consumidor, para isso, basta que cada consumidor leia a informação da fila.

C

No modelo publish/subscribe, assim que uma mensagem é enviada para a fila, ela é distribuída para o primeiro consumidor que fizer a requisição na fila.

D

No modelo publish/subscribe, os consumidores devem "assinar" a fila na qual desejam receber mensagens. No entanto, a assinatura somente funciona se nenhuma mensagem foi enviada para a fila.

E

Ambos os modelos ponto a ponto e publish/subscribe são similares, a diferença está no conteúdo da mensagem que é enviada.



A alternativa A está correta.

O modelo ponto a ponto tem uma característica marcante que é o "one-to-one", ou seja, um para um. Uma mensagem é criada pelo produtor e enviada a um único consumidor. Assim, se desejamos enviar a mesma informação por meio desse modelo, teremos que enviar a mensagem duas vezes: uma para cada consumidor.

Questão 2

Com relação à comunicação assíncrona por meio de um message broker, assinale a alternativa correta.

A

A utilização do message broker permite que os sistemas fiquem mais acoplados, ou seja, que um sistema tenha maior conhecimento sobre o outro, o que é uma vantagem na integração de sistemas.

B

O message broker geralmente é uma estrutura complexa, mas que permite o desacoplamento entre os sistemas e dá maior independência a cada sistema da integração.

C

O message broker é uma estrutura similar a um banco de dados, ou seja, o sistema A envia informações para serem armazenadas. O sistema B consome as informações e as exclui do message broker para permitir que novas informações sejam enviadas.

D

Na comunicação assíncrona, o serviço A aguarda que o processamento seja feito pelo serviço B para dar prosseguimento ao seu próprio processamento.

E

Existem dois tipos de message broker: os que utilizam filas e os que não utilizam filas.



A alternativa B está correta.

O message broker realmente é uma estrutura complexa e, exatamente por isso, diversos fornecedores possuem soluções que podem ser utilizadas na integração de sistemas. Isso faz com que o desenvolvimento de software para integração seja facilitado, pois a complexidade é assumida pelo fornecedor da solução. Se dois serviços A e B precisam se comunicar por meio de um message broker, eles precisam apenas saber o formato das mensagens recebidas e enviadas, trazendo um desacoplamento real que permite maior independência entre eles.

A API JMS e seus conceitos básicos

Confira os conceitos fundamentais da API JMS!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

O que é a API JMS?

O Java Message Service (JMS) é uma API Java que permite que os aplicativos criem, enviem, recebam e leiam mensagens. Essa API define um conjunto comum de interfaces que permitem que programas escritos na linguagem de programação Java se comuniquem com outras implementações de mensagens.



Atenção

A JMS é uma API Java que minimiza a quantidade de conceitos que você deve conhecer, mas permite que sejam desenvolvidas soluções sofisticadas de mensageria.

Ela admite uma comunicação fracamente acoplada e assíncrona. Mas existiu um esforço em desenvolver também uma API confiável, que evite a perda de mensagens ou o recebimento de mensagens duplicadas.

Quando utilizar a API JMS?

Grande parte dos servidores de aplicação Java suporta a API JMS nativamente, facilitando a vida do desenvolvedor de aplicações.

Algumas boas práticas de programação para soluções de grande porte (Java chama de aplicações corporativas – *enterprise applications*) já foram consideradas quando escolheram o JMS, pois uma aplicação corporativa não deve ter uma API fortemente acoplada e, por isso, você deve evitar o uso de soluções como o remote procedure call (RPC), ou chamada de procedimento remoto, nas seguintes situações:

- Deseja-se que os componentes não dependam de informações sobre as interfaces de outros componentes, para que os componentes possam ser facilmente substituídos.
- Deseja-se que o aplicativo seja executado independentemente de todos os componentes estarem ou não em execução simultaneamente.
- Deseja-se que o modelo de negócios do aplicativo permita que um componente envie informações para outro e continue operando sem receber uma resposta imediata.

Java tem a fama de ser uma linguagem com curva de aprendizado lenta, pois os desenvolvedores da linguagem incorporaram as mesmas diversas boas práticas e padrões de projeto de modo a direcionar o desenvolvedor a implementar de forma correta. Por isso, em um primeiro momento, pode parecer estranho que os desenvolvedores Java tenham já decidido incorporar o JMS nos servidores de aplicação de forma nativa, mas é porque eles sabem que aplicações corporativas comumente necessitam de soluções de mensageria.

Assim, quando a API JMS foi introduzida em 1998, seu propósito era permitir que aplicativos Java acessassem sistemas de middleware orientado a mensagens (MOM) já existentes, como o MQSeries da IBM. Desde então, muitos fornecedores adotaram e implementaram a API JMS, de modo que o JMS agora é uma solução completa de mensageria para soluções corporativas.

Conceitos básicos da API JMS

Uma **aplicação JMS** é composta das seguintes partes:

1

Provedor JMS

Um sistema de mensageria que implementa as interfaces JMS e fornece recursos administrativos e de controle. Uma implementação da plataforma Java EE inclui um provedor JMS nativamente.

2

Clientes JMS

Programas ou componentes, escritos na linguagem de programação Java, que produzem e consomem mensagens. Qualquer componente de aplicativo Java EE pode atuar como um cliente JMS.

3

Mensagens

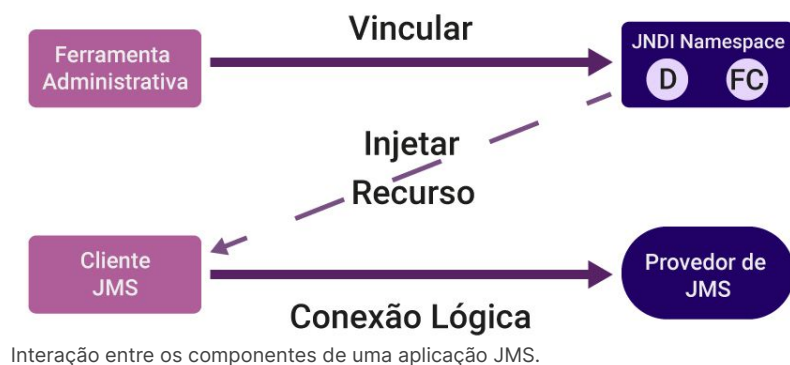
Os objetos que comunicam as informações entre clientes JMS.

4

Objetos administrados

Objetos JMS pré-configurados criados por um administrador para uso por clientes. Os dois tipos de objetos administrativos JMS são destinos (destinations) e fábricas de conexão (connection factories).

Veja a seguir a maneira como essas partes interagem. As ferramentas administrativas permitem que você vincule (bind) destinos e fábricas de conexão em um namespace Java naming and directory interface (JNDI). Basicamente, os componentes ganharão um “nome” e poderão ser encontrados por outros serviços.



Um cliente JMS pode então usar a injeção de recursos para acessar os objetos administrados no namespace criado e, em seguida, estabelecer uma conexão lógica com os mesmos objetos por meio do provedor JMS.

Basicamente, esse processo permite que objetos, destinos e fábricas de conexão sejam utilizados por clientes JMS! Você não precisa se preocupar com os detalhes relacionados ao namespace, JNDI e injeção de recursos!

Domínios de mensageria

Antes da existência da API JMS, a maioria dos produtos de mensageria suportava tanto a abordagem ponto a ponto como a publish/subscribe. A especificação JMS fornece um domínio separado para cada abordagem e define uma conformidade para cada domínio.

Assim, um provedor de JMS pode implementar um ou ambos os domínios. Um provedor Java EE (Enterprise Edition) deve implementar ambos os domínios. Na verdade, a maioria das implementações da API JMS suporta ambos os domínios, e alguns clientes JMS utilizam ambos os domínios em um único aplicativo. Dessa forma, a API JMS fornece maiores flexibilidades aos aplicativos de mensageria.



Comentário

Além disso, a especificação JMS vai um passo além: ela fornece interfaces comuns que permitem usar a API JMS de uma forma que não é específica para nenhum dos domínios. Ou seja, você desenvolverá códigos muito similares para ambos os domínios! O que é ideal, sem precisar decorar detalhes de implementação de um domínio ou de outro.

Assim é possível usar um mesmo código para enviar e receber mensagens no domínio ponto a ponto ou publish/subscribe. Os destinos que você usa permanecem específicos do domínio e o comportamento do aplicativo dependerá em parte se você estiver usando uma fila (ponto a ponto) ou um tópico (publish/subscribe). Já o seu código é o mesmo a ambos os domínios, tornando seus aplicativos flexíveis e reutilizáveis.

Principais conceitos da API JMS

Confira os principais conceitos que envolvem a API JMS!

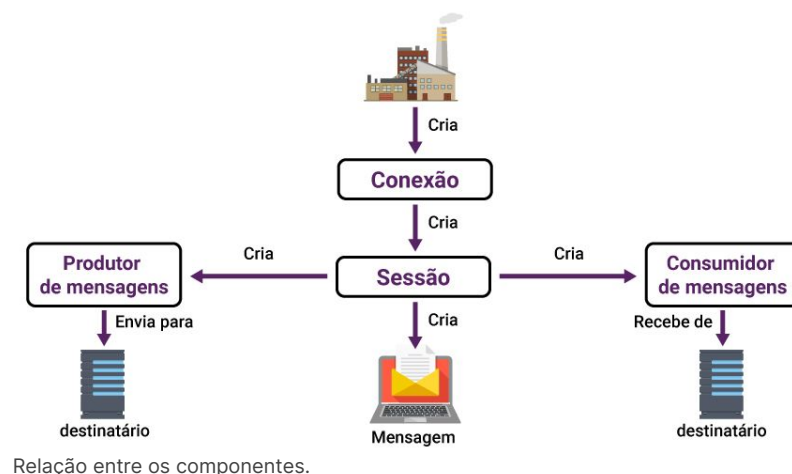


Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

O modelo de programação da API JMS (parte 1)

Vamos agora nos aprofundar um pouco nos blocos de construção básicos de um aplicativo JMS.



Podemos indicar os blocos de construção básicos da seguinte forma:

- Objetos administrativos: fábricas de conexão e destinos
- Conexões
- Sessões
- Produtores de mensagens
- Consumidores de mensagens
- Mensagens

Objetos administrativos

Existem dois tipos de objetos administrativos: **destinos** e **fábricas de conexão**. Eles são melhor mantidos administrativamente em vez de programaticamente: isso significa que você realizará algumas configurações ao invés de definir parâmetros via código!

A forma de configurar esses objetos varia de implementação para implementação, mas apresentaremos a seguir alguns exemplos concretos.

Os clientes JMS acessam esses objetos por meio de interfaces e, portanto, um aplicativo cliente pode ser executado com pouca ou nenhuma alteração em mais de uma implementação da API JMS. Normalmente, um administrador configura objetos administrativos em um namespace do JNDI e os clientes JMS os acessam usando injeção de recursos.

Por exemplo, no **servidor de aplicação GlassFish**, você pode utilizar o seguinte comando:

```
java
$ asadmin create-jms-resource
```

Alternativamente, você também pode acessar o Administration Console para criar estes mesmos objetos administrados JMS na forma de recursos de conector ou pode especificar os recursos em um arquivo chamado **GlassFish-resources.xml** que pode ser agrupado com um aplicativo.

Fábricas de conexão JMS

Uma fábrica de conexão é um padrão de projeto na forma de um objeto que é utilizado pelo cliente para criar a conexão para um provedor.



Comentário

Na verdade, internamente, a fábrica de conexão encapsula um conjunto de parâmetros de configuração definido pelo administrador (ao criar o recurso).

Cada conexão é uma instância de interface **ConnectionFactory**, **QueueConnectionFactory** (ponto a ponto) ou **TopicConnectionFactory** (publish/subscribe).

No início de um programa cliente JMS, você normalmente injeta um recurso de fábrica de conexão no objeto **ConnectionFactory**. Por exemplo, o trecho de código a seguir especifica um recurso que tem o nome JNDI "jms/ConnectionFactory" e o atribui a um objeto **ConnectionFactory**. Vejamos:

```
java
@Resource(lookup = "jms/ConnectionFactory")
private static ConnectionFactory connectionFactory;
```

A primeira linha é onde acontece a injeção de recursos e a segunda cria uma fábrica de conexões “apontando” para esse recurso.

Destinos JMS



Atenção

Um destino (destination) é o objeto que o cliente utiliza para especificar o destino das mensagens que ele produz e a fonte das mensagens que ele consome. No domínio de mensageria ponto a ponto, os destinos são chamados de filas. No domínio de mensageria publish/subscribe, os destinos são chamados de tópicos. Uma aplicação JMS pode utilizar múltiplas filas ou tópicos, ou ambos!

Para criar um destino usando o servidor de aplicação GlassFish, você precisa apenas criar um recurso JSM destino que especifica um nome JNDI para o destino!

A seguir, temos um exemplo do comando do **GlassFish** para criar uma fila chamada **PhysicalQueue**:

```
java
asadmin> create-jmsdest --desttype queue --property User=public:Password=public
PhysicalQueue
Command create-jmsdest executed successfully.
```

Na implementação do servidor de aplicação GlassFish, cada recurso de destino refere-se a um destino físico. Você pode criar um destino físico explicitamente, mas se não o fizer, o servidor de aplicação cria quando necessário e o exclui quando você exclui o recurso de destino.

Além de injetar um recurso de fábrica de conexão em um programa cliente, você normalmente injeta um recurso de destino. Ao contrário das fábricas de conexão, os destinos são específicos para um domínio ou outro. Para criar um aplicativo que permite usar o mesmo código para tópicos e filas, você atribui o destino a um objeto Destination.

O código a seguir especifica dois recursos: uma **fila** e um **tópico**.

```
java
@Resource(lookup = "jms/Queue")
private static Queue queue;

@Resource(lookup = "jms/Topic")
private static Topic topic;
```

Com as interfaces disponíveis, você pode misturar ou combinar fábricas de conexão e destinos. Ou seja, além de usar a interface ConnectionFactory, você pode injetar um recurso QueueConnectionFactory e usá-lo com um Topic, e pode injetar um recurso TopicConnectionFactory e usá-lo com um Queue.

O modelo de programação da API JMS (parte 2)

Programando com a API JMS

Confira como programar utilizando a API JMS!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Conexões JMS

Uma conexão encapsula uma conexão virtual com um provedor JMS. Por exemplo, uma conexão (que é um objeto) pode representar uma conexão TCP/IP aberta entre um cliente e um daemon de serviço do provedor. Ou seja, você usa uma conexão para abrir o canal de comunicação com o provedor!

As conexões implementam a interface de Connection. Quando você tem um objeto ConnectionFactory, você pode usá-lo para criar um objeto Connection.

```
java
```

```
Connection connection = connectionFactory.createConnection();
```

Mas atenção, antes que um aplicativo seja concluído, você deve fechar todas as conexões criadas. A falha ao fechar uma conexão pode fazer com que os recursos não sejam liberados pelo provedor JMS. Fechar uma conexão também fecha suas sessões e seus produtores e consumidores de mensagens.

```
java
```

```
connection.close();
```

Outro detalhe importante: antes que sua aplicação consuma mensagens, você deve chamar o método start da sua conexão. Se você quiser parar de enviar temporariamente mensagens sem fechar a conexão, use o método stop.

Sessões JMS

Uma sessão é um contexto de thread único para produzir e consumir mensagens, sendo usada para criar:

- Produtores de mensagens
- Consumidores de mensagens
- Mensagens
- Navegadores de fila
- Filas e tópicos temporários

Uma sessão fornece um contexto transacional onde se agrupa um conjunto de envios e recebimentos em uma unidade atômica de trabalho, similar ao conceito em banco de dados: toda a **sequência de instruções** dentro de uma transação é considerada única!

As sessões implementam a interface Session. Depois de criado o objeto Connection, você irá utilizá-lo para criar o objeto Session.

```
java
```

```
Session session = connection.createSession(false, Session.AUTO_ACKNOWLEDGE);
```

O primeiro argumento significa que a sessão não é transacional; o segundo significa que a sessão reconhece automaticamente as mensagens quando elas são recebidas com sucesso.

Como exemplo, para criar uma sessão transacional, faríamos:

```
java
```

```
Session session = connection.createSession(true, 0);
```

Aqui, o primeiro argumento significa que a sessão é transacional; o segundo indica que o reconhecimento de mensagem não está especificado para sessões transacionadas.

Não se apegue aos detalhes, o importante é você entender uma visão geral de cada objeto e seu uso no contexto de uma implementação de solução de mensageria.

Produtor de mensagens JMS

Um produtor de mensagens é um objeto criado pela sessão e utilizado para enviar mensagens para um destinatário. Ele implementa a interface `MessageProducer`.

Você usa o objeto `Session` para criar um `MessageProducer` para um destino. A seguir, temos alguns exemplos que mostram como criar um produtor para um objeto `Destination`, `Queue` e `Topic`.

```
java
```

```
MessageProducer producer = session.createProducer(dest);  
MessageProducer producer = session.createProducer(queue);  
MessageProducer producer = session.createProducer(topic);
```

Você pode criar um produtor não identificado, ou seja, utilizando `null` como argumento. Nesse caso, você somente especificará o destino quando for enviar a mensagem. Seria algo assim:

```
java
```

```
producer.send(message);
```

Outro exemplo:

```
java
```

```
MessageProducer a = session.createProducer(null);  
a.send(dest, message);
```

Nesse caso, quando chamamos o método `send`, especificamos o destino e a mensagem a ser enviada.

Consumidor de mensagens JMS

Um consumidor de mensagem é um objeto criado por uma `Session` e usado para receber mensagens enviadas a um destino e implementar a interface `MessageConsumer`.

Um consumidor de mensagem permite que um cliente JMS registre interesse em um destino com um provedor JMS. O provedor JMS gerencia a entrega de mensagens de um destino para os consumidores registrados do destino.

Similar ao produtor de mensagens, temos os seguintes exemplos:

```
java
```

```
MessageConsumer consumer = session.createConsumer(dest);  
MessageConsumer consumer = session.createConsumer(queue);  
MessageConsumer consumer = session.createConsumer(topic);
```

Você usa o método **Session.createDurableSubscriber** para criar um assinante de tópico “durável”. Esse método só é válido se você estiver usando um Topic.

Depois de criar um consumidor de mensagem, ele se torna ativo e você pode usá-lo para receber mensagens, porém a entrega de mensagens não começa até que você inicie a conexão chamando o **método start**. Para tornar o consumidor de mensagem inativo, é necessário empregar o método close.

Você usa o **método receive** para consumir uma mensagem de forma síncrona e pode utilizar esse método a qualquer momento depois de chamar o método start:

```
java
```

```
connection.start();  
Message m = consumer.receive();  
connection.start();  
Message m = consumer.receive(1000);
```

O parâmetro 1000 na última linha é o timeout.

O modelo de programação da API JMS (parte 3)

Listeners JMS

O listener de mensagens é um objeto que atua como um manipulador de eventos assíncrono para mensagens. Esse objeto implementa a interface `MessageListener`, que contém um **método, `onMessage`**, em que você define as ações a serem tomadas quando uma mensagem chega.



Comentário

Você registra o listener de mensagens com um `MessageConsumer` específico usando o método `setMessageListener`.

Por exemplo, se você definir uma classe chamada `listener` que implementa a interface `MessageListener`, você pode registrar o listener de mensagens da seguinte forma:

```
java
```

```
Listener myListener = new Listener();  
consumer.setMessageListener(myListener);
```

Depois de registrar o listener de mensagens, você chama o método `start` no `Connection` para iniciar a entrega de mensagens.



Atenção

Se você chamar o `start` antes de registrar o listener de mensagens, provavelmente você perderá mensagens!

Quando a entrega da mensagem começa, o provedor JMS chama automaticamente o método `onMessage` do `Listener` da mensagem. O método `onMessage` recebe um argumento do tipo `Message`, em que sua implementação do método pode converter em qualquer um dos outros tipos de mensagem.

Exemplo de uma fila

Agora que tivemos uma visão geral dos principais componentes, classes e interfaces do JMS, vamos fazer um exemplo de uma fila (`Queue`). Isso vai te ajudar a entender melhor cada um dos componentes citados.

Criando os recursos

Para criar uma fila, primeiro é preciso instalar um servidor de aplicação. No nosso caso, instalamos o `GlassFish`. Agora, criaremos **dois JNDI**:

1. Uma fábrica de conexão chamada de `myQueueConnectionFactory`.
2. Um recurso `destination` chamado `myQueue`.

Vejamos o passo a passo de como criar a fábrica de conexão e a `destination`:

- Abra o console de administrador (geralmente `localhost:4848`) e vá em `JMS Resource` → `Connection Factories` → `New`.
- Defina `pool name` como `myQueueConnectionFactory` e `Resource Type` como `QueueConnectionFactory`. Clique em `OK`.
- Vá em `JMS Resource` → `Destination Resources` → `New`. Coloque `JNDI Name` como `myQueue`. `Physical Destination Name` pode ser `abc` e `Resource Type` `javax.jms.Queue`. Clique `OK`.
- Pronto, a fábrica de conexão e a `destination` estão criadas.

Código do remetente

A seguir, temos o código do remetente da mensagem. Criamos inicialmente um objeto `ctx` do tipo `Context` para que possa fazer uma procura pelo recurso que criamos no servidor da aplicação `GlassFish`. Alternativamente, poderíamos utilizar a injeção de recursos.

```

java

import java.io.BufferedReader;
import java.io.InputStreamReader;
import javax.naming.*;
import javax.jms.*;

public class MySender {
    public static void main(String[] args) {
        try
        {
            InitialContext ctx=new InitialContext();
            QueueConnectionFactory f=(QueueConnectionFactory)
ctx.lookup("myQueueConnectionFactory");
            QueueConnection con=f.createQueueConnection();
            con.start();

            QueueSession ses=con.createQueueSession(false, Session.AUTO_ACKNOWLEDGE);

            Queue t=(Queue)ctx.lookup("myQueue");
            QueueSender sender=ses.createSender(t);
            TextMessage msg=ses.createTextMessage();
            String s="Minha Mensagem!";
            msg.setText(s);
            sender.send(msg);
            System.out.println("Mensagem enviada com sucesso");
        }

        con.close();
    }catch(Exception e){System.out.println(e);}
}

```

Na linha 11, criamos uma instância de nossa fábrica de conexões (myQueueConnectionFactory) e, a partir dela, criamos nossa conexão com a fila (linha 12). Na linha 13, iniciamos a conexão. Reveja a imagem Relação entre os componentes! Ela vai fazer mais sentido agora!

Na linha 15, criamos a sessão que reconhece automaticamente as mensagens quando elas são recebidas com sucesso (segundo argumento). Nas linhas 17, 18 e 19 criamos uma fila (Queue) a partir do recurso destination que criamos no GlassFish, um remetente de fila (QueueSender) e um TextMessage (é um tipo de mensagem).

Aqui, novamente, poderíamos ter utilizado a injeção de recursos ao invés do lookup da linha 17.

Criamos uma String com a mensagem desejada (linha 20). Na linha 21, definimos a mensagem utilizando o método setText de msg (que é uma TextMessage). Finalmente, enviamos a mensagem utilizando o método send do QueueSender (linha 22).

Código do receptor

A seguir, temos o código do receptor da mensagem. Criamos inicialmente um objeto ctx do tipo Context (linha 6) para que possa fazer uma procura pelo recurso myQueueConnectionFactory. Criamos a conexão a partir desse recurso (linha 8) e iniciamos a conexão (linha 9).

```

java
import javax.jms.*;
import javax.naming.InitialContext;
public class MyReceiver {
    public static void main(String[] args) {
        try{
            InitialContext ctx=new InitialContext();
            QueueConnectionFactory f=(QueueConnectionFactory)
ctx.lookup("myQueueConnectionFactory");
            QueueConnection con=f.createQueueConnection();
            con.start();

            QueueSession ses=con.createQueueSession(false, Session.AUTO_ACKNOWLEDGE);

            Queue t=(Queue)ctx.lookup("myQueue");
            QueueReceiver receiver=s ses.createReceiver(t);
            MyListener listener=new MyListener();
            receiver.setMessageListener(listener);
            System.out.println("Receptor pronto!");
            System.out.println("press Ctrl+c to shutdown...");
            while(true){
                Thread.sleep(1000);
            }
        }catch(Exception e){System.out.println(e);}
    }
}

```

Na linha 11, criamos a sessão a partir de conexão con. Criamos uma fila (Queue) a partir do recurso que criamos no GlassFish chamado MyQueue (linha 13).

Na linha 14, criamos um receiver, que nada mais é do que um receptor de mensagens a partir da nossa sessão (ses) e passando como parâmetro a Fila (neste caso t). Nas linhas 15 e 16, criamos um listener e registramos em nosso receptor receiver.

Pronto, agora o programa fica em loop infinito (linhas 19 a 21), aguardando uma mensagem. Quando uma mensagem for recebida, o método onMessage do nosso listener (classe MyListener a seguir) será acionado automaticamente.

Vejamos agora o código da nossa implementação do myListener. Nela, temos definido o método onMessage que recebe como parâmetro um objeto de Message.

```

java
import javax.jms.*;
public class MyListener implements MessageListener {

    public void onMessage(Message m) {
        try{
            TextMessage msg=(TextMessage)m;

            System.out.println("Mensagem recebida:"+msg.getText());
        }catch(JMSEException e){System.out.println(e);}
    }
}

```

Na linha 6, é feito um casting, ou seja, convertemos o objeto do tipo Message para TextMessage (ver código do remetente). Na linha 8, apresentamos na tela a mensagem recebida utilizando o método getText() do objeto msg.

Verificando o aprendizado

Questão 1

A API JMS deve ser utilizada

A

quando os componentes que devem ser integrados possuem um forte acoplamento.

B

quando todos os componentes a serem integrados estão em execução.

C

quando um componente envia uma informação para o outro e pode continuar operando.

D

quando os componentes utilizam diversas linguagens de programação como Java, PHP, Python etc.

E

quando a quantidade de componentes a ser integrada é maior que 3.



A alternativa C está correta.

A API JMS é baseada na utilização de um provedor JMS, ou seja, um componente intermediário que recebe mensagens do produtor e as mantém em uma fila até que o consumidor as solicite. É ideal quando o produtor (componente) pode continuar operando sem a necessidade de aguardar a resposta do consumidor (componente).

Questão 2

Com relação ao modelo de programação da API do JMS, assinale a alternativa correta.

A

Os objetos administrativos são as fábricas de conexão e as sessões.

B

Os destinos podem ser do tipo fila ou tópico.

C

As conexões JMS realizam a comunicação entre o consumidor e o produtor da mensagem.

D

Existe somente um tipo de mensagem na API JMS que é a `TextMessage`.

E

O listener é utilizado somente na comunicação síncrona.



A alternativa B está correta.

Existem dois tipos de destinations: a fila (queue), quando o domínio escolhido é o ponto a ponto, e o tópico, quando o domínio escolhido é o publish/subscribe. No primeiro caso, a mensagem será enviada para um único consumidor e, no segundo caso, para todos os consumidores que assinaram aquele tópico (topic).

Introdução ao RabbitMQ

Confira o que é o RabbitMQ e como utilizá-lo!



Conteúdo interativo

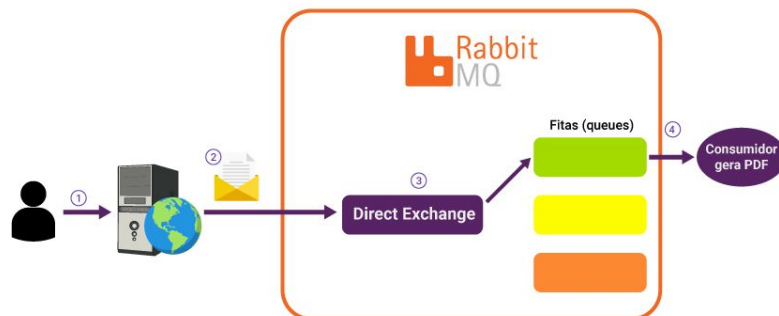
Acesse a versão digital para assistir ao vídeo.

O RabbitMQ é um software de fila de mensagens de código aberto escrito em Erlang. Ele suporta vários protocolos como AMQP, STOMP, MQTT e HTTP e pode ser executado em diferentes ambientes de nuvem e sistemas operacionais.

O produtor e consumidor de mensagens do RabbitMQ podem ser desenvolvidos em linguagens de programação como Python e Java.

Funcionamento geral do RabbitMQ

Imagine um aplicativo Web que permite usuários enviarem informações e que, em seguida, essas informações gerem um PDF. Se essa capacidade de gerar o PDF fosse realizada por outro servidor, o RabbitMQ poderia funcionar como forma de comunicação entre o servidor Web e o servidor que gera o PDF, veja:



Visão geral do uso do RabbitMQ.

Agora confira o passo a passo representado pelos números ilustrados na imagem:

1. Um usuário envia suas informações que serão usadas para gerar o PDF para o aplicativo da Web.
2. O aplicativo Web (produtor), envia uma mensagem ao RabbitMQ que contém as informações do usuário e que é para gerar o PDF.
3. Dentro do RabbitMQ, uma exchange (que recebe mensagens e as encaminha), recebe a mensagem e a direciona para a fila certa. Essa exchange é um conceito novo implementado no RabbitMQ.
4. Um consumidor (que se chama Gera PDF) recupera a mensagem da fila e começa a processar o PDF.

Instalação do RabbitMQ

Veja agora os passos para a instalação do RabbitMQ em um sistema Ubuntu. No geral, a instalação é bastante simples.

Como super usuário, atualize o sistema e instale os pacotes necessários.

```
plain-text

# apt-get update
# apt-get install apt-transport-https wget gnupg curl
```

Em seguida, instale o servidor.

```
plain-text

# apt-get install rabbitmq-server
```

Agora, habilitaremos o serviço de forma que ele inicie automaticamente durante o boot.

```
plain-text

# systemctl enable rabbitmq-server
```

Inicializaremos o serviço.

```
plain-text

# systemctl start rabbitmq-server
```

Para verificar se tudo está correto, use o comando **netstat** e busque informações na **porta 5672**, veja:

```
plain-text

# netstat -nl | grep -E -i "proto|5672"
```

Em seguida, você irá obter a seguinte saída:

```
plain-text

Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp      0      0 0.0.0.0:25672           0.0.0.0:*              LISTEN
tcp6     0      0 :::5672                 :::*                    LISTEN
```

Configuração do RabbitMQ

Para configurar a interface Web do RabbitMQ, vamos criar um usuário admin, dar permissão de administrador para ele e ajustar as permissões. Utilizaremos o comando **rabbitmqctl**.

```
plain-text

# rabbitmqctl add_user admin 123
# rabbitmqctl set_user_tags admin administrator
# rabbitmqctl set_permissions -p / admin ".*" ".*" ".*"
```

Vamos habilitar o **plugin de gerenciamento** do RabbitMQ.

```
plain-text

# rabbitmq-plugins enable rabbitmq_management
```

A saída desse comando será algo similar a:

```
plain-text

Enabling plugins on node rabbit@ubuntu20:
rabbitmq_management
The following plugins have been configured:
  rabbitmq_management
  rabbitmq_management_agent
  rabbitmq_web_dispatch
Applying plugin configuration to rabbit@ubuntu20...
The following plugins have been enabled:
  rabbitmq_management
  rabbitmq_management_agent
  rabbitmq_web_dispatch

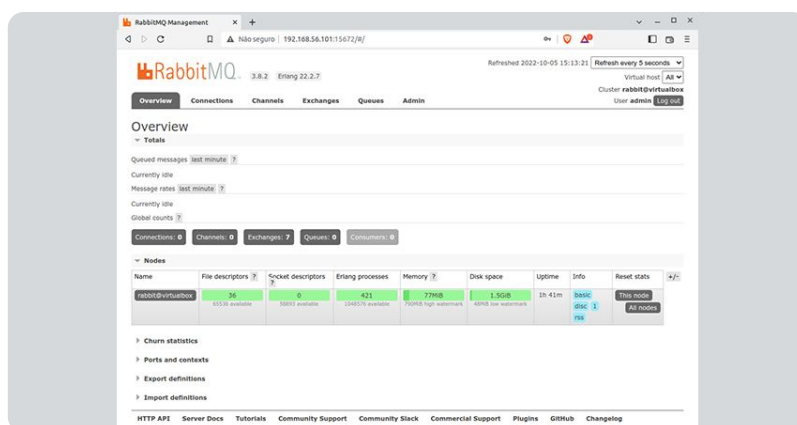
started 3 plugins.
```

Agora podemos acessar a interface Web. Abra seu navegador e coloque o IP da instalação do RabbitMQ (aqui utilizaremos 192.168.56.101) na porta 15672 e aparecerá a seguinte tela de autenticação:



Tela de autenticação do RabbitMQ.

Autentique-se no sistema com username admin e senha 123 e veja a tela inicial da interface Web do RabbitMQ após a autenticação:



Tela inicial da interface Web do RabbitMQ

Para instalar em outros ambientes, como em containers, sistema operacional Windows, outras distribuições de Linux, consulte o site do RabbitMQ.

O RabbitMQ é uma solução comumente utilizada, com as seguintes vantagens:

- Reconhecimentos e confirmações de entrega de mensagens que aumentam a confiabilidade da fila de mensagens, reduzindo a perda de mensagens.
- Roteamento flexível que permite a entrega de mensagens específicas a filas específicas e consumidores específicos (lembre-se de que quem recebe as mensagens são as exchange).
- Vários tipos de exchanges que permitem diferentes maneiras de direcionar mensagens para os consumidores
- Facilmente implantado em nuvens corporativas e públicas porque é leve.

Devemos ainda considerar algumas desvantagens do RabbitMQ:

- Aspectos muito técnicos, especialmente para iniciantes.
- Incapacidade de visualizar as filas e as mensagens na fila.
- Mensagens de erro vagas, que dificultam o diagnóstico de falhas.

Desenvolvendo um produtor/consumidor em Python para RabbitMQ

Confira os métodos para desenvolver um produtor/consumir utilizando Python para RabbitMQ!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Olá, mundo em Python!

Como em todo “bom” exemplo de código em linguagem de programação, construiremos uma aplicação em Python bem simples. Para isso, é preciso instalar uma biblioteca que dê suporte à comunicação com um servidor RabbitMQ.

```
plain-text
```

```
$ pip install pika --upgrade
```

Essa biblioteca facilita o desenvolvimento de produtores e consumidores de mensagens ao abstrair para o programador grande parte dos detalhes da comunicação com o servidor RabbitMQ.

Produtor em Python

O objetivo do código `send.py`, apresentado a seguir, é enviar uma mensagem de “Ola Mundo!” para o servidor RabbitMQ.

```
python

#!/usr/bin/env Python
import pika

credentials = pika.PlainCredentials('admin', '123')
connection = pika.BlockingConnection(pika.ConnectionParameters('192.168.56.101',
5672, '/', credentials))

channel = connection.channel()

channel.queue_declare(queue='hello')

channel.basic_publish(exchange='', routing_key='hello', body='Ola Mundo!')

print(" [x] Enviado 'Ola Mundo!'")
connection.close()
```

Na linha 4, criamos um objeto com as credenciais de acesso, as mesmas credenciais já utilizadas. Na linha 5, criamos uma conexão com o Rabbit, passando como parâmetros o IP do servidor RabbitMQ (**192.168.56.101**), a porta padrão (5672), o virtual host (não se preocupe com isso!), e o objeto credentials já criado.

A partir da conexão criada, criamos um objeto channel, que é uma classe que fornece um wrapper para interagir com o RabbitMQ implementando os métodos e comportamentos para um canal que utiliza o protocolo AMQP. Em seguida, antes de enviar, precisamos garantir que a fila de destinatários exista. Se enviarmos uma mensagem para um local não existente, o RabbitMQ apenas descartará a mensagem. Criamos uma fila “hello” para a qual a mensagem será entregue (linha 9).

Nesse ponto, estamos prontos para enviar uma mensagem. Nossa primeira mensagem conterá apenas uma string “Ola Mundo!” e queremos enviá-lo para nossa fila “hello”. No RabbitMQ uma mensagem nunca pode ser enviada diretamente para a fila, ela sempre precisa passar por uma exchange.

O que precisamos saber agora é como utilizar a exchange default identificada por uma string vazia. Assim, na linha 11, definimos para qual exchange a mensagem irá, a fila e a mensagem (body). Na linha 14 fechamos a conexão.

Consumidor em Python

O objetivo do código recebe.py, apresentado a seguir, é consumir mensagens que estejam no exchange padrão e na fila “hello” do servidor RabbitMQ.

```
python

#!/usr/bin/env Python
import pika, sys, os

def main():
    credentials = pika.PlainCredentials('admin', 'kamisama123')
    connection = pika.BlockingConnection(pika.ConnectionParameters('192.168.56.101',
5672, '/', credentials))

    channel = connection.channel()
    channel.queue_declare(queue='hello')

    def callback(ch, method, properties, body):
        print(" [x] Recebido %r" % body)

    channel.basic_consume(queue='hello',
        on_message_callback=callback, auto_ack=True)

    print(' [*] Aguardando Mensagens. Para sair pressione CTRL+C')
    channel.start_consuming()

if __name__ == '__main__':
    try:
        main()
    except KeyboardInterrupt:
        print('Interrupted')
        try:
            sys.exit(0)
        except SystemExit:
            os._exit(0)
```

Inicialmente, definimos uma função main (linha 4) que será chamada por padrão (linha 22). Da linha 5 a linha 9, seguimos os mesmos passos já vistos no código em Python do produtor de mensagens. Ou seja, criamos um objeto credentials com as credenciais de acesso ao servidor RabbitMQ. Criamos um objeto connection com os parâmetros IP, porta, virtual host e credenciais e que faz a conexão com o servidor.

A partir do objeto de conexão, criamos um canal e informamos a fila de interesse, nesse caso, “hello”.



Atenção

Agora definimos uma função de callback (os parâmetros dessa função são padrão) que será executado quando a fila receber alguma mensagem a ser consumida (linha 11 e 12). As informações de mensagem da fila serão atribuídas ao parâmetro body.

A seguir, definimos no objeto channel a fila a ser consumida e qual função deve ser chamada quando uma mensagem chegar na fila (on_message_callback). Nesse caso, a função se chama callback.

Finalmente, chamamos channel.start_consuming() para fazer com que o código fique em loop infinito verificando se alguma mensagem chega na fila do servidor RabbitMQ.

Executando os códigos em Python

Agora, colocaremos tudo para funcionar. Inicialmente, execute o **receive.py** para que o consumidor de mensagens fique monitorando a fila “hello”.

```
plain-text

$ Python receive.py
[*] Aguardando Mensagens. Para sair pressione CTRL+C
```

Em outro terminal, execute o send.py:

```
plain-text

$ Python send.py
[x] Enviado 'Ola Mundo!'
```

Nesse momento, se observarmos o terminal do consumidor, teremos:

```
plain-text

$ Python receive.py
[*] Aguardando Mensagens. Para sair pressione CTRL+C
[x] Recebido b'Ola Mundo!'
```

Você pode ainda testar enviar a mensagem “Ola Mundo!” mais vezes, observe:

```
plain-text

$ Python send.py
[x] Enviado 'Ola Mundo!'
$ Python send.py
[x] Enviado 'Ola Mundo!'
$ Python send.py
[x] Enviado 'Ola Mundo!'
```

```
plain-text

$ Python receive.py
[*] Aguardando mensagens. Para sair pressione CTRL+C
[x] Recebido b'Ola Mundo!'
[x] Recebido b'Ola Mundo!'
[x] Recebido b'Ola Mundo!'
```

Esse é um exemplo simples, mas ele demonstra como a biblioteca pika e o servidor RabbitMQ abstraem boa parte da complexidade técnica de um servidor de mensageria, deixando a cargo do programador apenas a lógica do negócio a ser modelada.

Desenvolvendo um produtor/consumidor em Java para RabbitMQ

Confira os métodos para desenvolver um produtor/consumidor para RabbitMQ utilizando Java!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Olá, mundo em Java!

Assim como em outras linguagens de programação, o Java dá suporte ao RabbitMQ utilizando a biblioteca **amqp-client**. Você pode usar o Maven ou o Gradle como ferramenta de gerenciamento de dependências, em ambas é possível configurar para que essa biblioteca possa ser inserida em seu projeto.

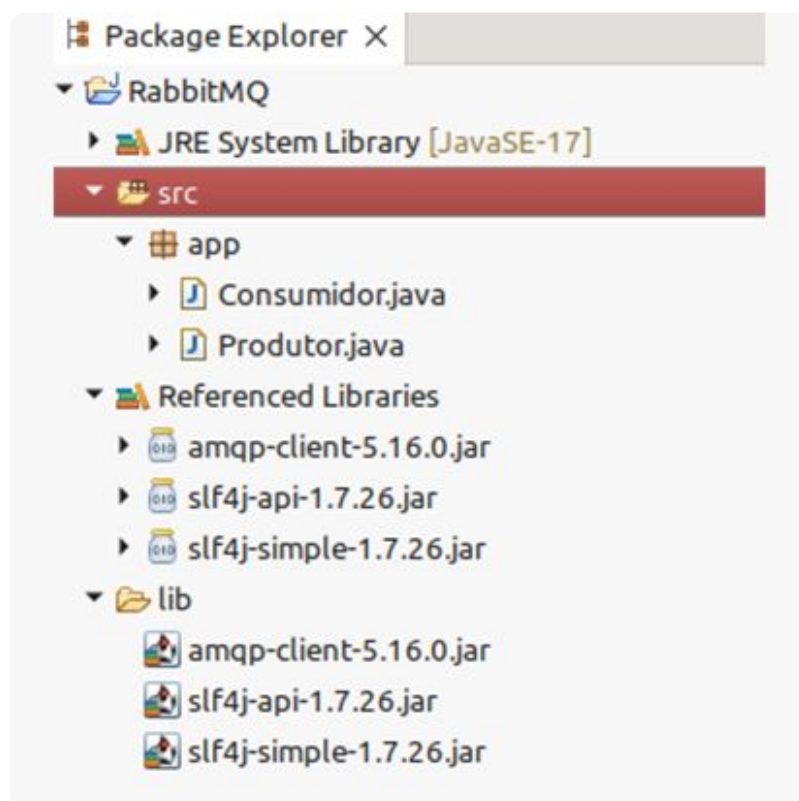
Baixar o arquivo jar na versão 5.16.0 de um repositório na Web.



Atenção

A biblioteca amqp-client depende de outras duas bibliotecas: SLF4J API e SLF4J Simple que podem ser baixadas de repositórios na Web.

No projeto do eclipse, adicione os 3 arquivos .jar no projeto de forma que façam parte das bibliotecas referenciadas, conforme imagem a seguir. Se você utilizar outra IDE de desenvolvimento em Java, basta que seu projeto faça referências a esses 3 arquivos jar.



Estrutura do projeto no Eclipse.

Pronto! Estamos preparados para desenvolver o código dos arquivos Produtor.java (que enviará mensagem para a fila) e o Consumidor.java (que consumirá mensagem da fila).

Produtor em Java

O objetivo do código Produtor.java é enviar uma mensagem de “Ola Mundo!” para o nosso servidor RabbitMQ! Veja:

```
java

package app;
import com.rabbitmq.client.ConnectionFactory;
import com.rabbitmq.client.Connection;
import com.rabbitmq.client.Channel;
import java.io.IOException;
import java.util.concurrent.TimeoutException;

public class Produtor {

    public static void main(String[] args) {
        ConnectionFactory factory = new ConnectionFactory();
        factory.setUsername("admin");
        factory.setPassword("123");
        factory.setHost("192.168.56.101");
        try (Connection connection = factory.newConnection()) {
            Channel channel = connection.createChannel();
            channel.queueDeclare("hello", false, false, false, null);
            String mensagem = "Ola Mundo!";
            channel.basicPublish("", "hello", null, mensagem.getBytes("UTF-8"));

        } catch (IOException | TimeoutException e) {
            e.printStackTrace();
        }
    }
}
```

Nas linhas 1 a 6, definimos o pacote de nosso código e importamos as classes necessárias para comunicar com o RabbitMQ, além de classes de exceção (erros) que devem ser tratados em Java.

Nas linhas 11 a 14, criamos a factory e configuramos ela com os parâmetros, como o username e password do servidor RabbitMQ (linhas 12 e 13), e o endereço IP do servidor (linha 14). Na linha 15, tentamos abrir uma conexão com o servidor RabbitMQ, se houver alguma falha, a exceção será tratada nas linhas 21 a 22. Se tudo der certo, conseguimos a conexão com o servidor e o objeto connection representa essa conexão.

Na linha 16, criamos um objeto channel a partir de conexão e na linha 17 informamos que a mensagem será enviada para a fila “hello”. Criamos a string que contém a mensagem (linha 18). Na linha 19, enviamos a mensagem para a exchange default, que é representada por “” no primeiro parâmetro do método basicPublish de channel.

Pronto! O código deve ser capaz de enviar a mensagem “Ola Mundo” para a exchange default, fila “hello” do nosso servidor RabbitMQ.

Consumidor em Java

O objetivo é desenvolver um código que consuma mensagens da fila “hello” do nosso servidor RabbitMQ, veja:

```

java

package app;
import com.rabbitmq.client.ConnectionFactory;
import com.rabbitmq.client.Consumer;
import com.rabbitmq.client.DefaultConsumer;
import com.rabbitmq.client.Envelope;
import com.rabbitmq.client.Connection;
import java.io.IOException;
import java.util.concurrent.TimeoutException;
import com.rabbitmq.client.AMQP;
import com.rabbitmq.client.Channel;
public class Consumidor {
    public static void main(String[] args) {
        ConnectionFactory factory = new ConnectionFactory();
        factory.setUsername("admin");
        factory.setPassword("123");
        factory.setHost("192.168.56.101");
        try (Connection connection = factory.newConnection()) {
            Channel channel = connection.createChannel();
            channel.queueDeclare("hello", false, false, false, null);
            Consumer consumer = new DefaultConsumer(channel) {
                @Override
                public void handleDelivery(String consumerTag, Envelope envelope,
AMQP.BasicProperties properties, byte[] body) throws IOException {
                    String mensagem = new String(body, "UTF-8");
                    System.out.println("Mensagem recebida: " + mensagem);
                }
            };
            channel.basicConsume("hello", true, consumer);
        } catch (IOException | TimeoutException e) {
            e.printStackTrace();
        }
    }
}

```

O código pode parecer complexo, mas a complexidade é da linguagem Java, e não do uso da amqp-client! Nas linhas 1 a 10, definimos o pacote de nossa classe e realizamos os imports necessários. Nas linhas 13 a 16, criamos um objetivo connection que representa a conexão com o servidor RabbitMQ e configuramos os parâmetros relacionados com o servidor.

Na linha 17, tentamos realizar a conexão com o servidor. Caso aconteça algum problema, o erro será tratado nas linhas 28 a 30. Na linha 18 e 19, criamos um objeto channel que informa ao servidor que utilizará a file “hello”.

Agora sim, a partir da **linha 18** temos a parte mais complexa do Java.

Na linha 18, criamos um objeto consumer a partir de channel. As chaves que abrimos na linha 20 e fechamos na linha 26 informam ao Java que faremos “algo” nesse objeto da classe Consumer. Esse “algo” nada mais é que sobrescrever o método handleDelivery, que é o método chamado automaticamente quando uma mensagem chega na exchange default, fila “hello” do nosso servidor.

Basicamente, na implementação do método handleDelivery (linha 23 e 24), estamos recuperando a mensagem (que foi passada por parâmetro para handleDelivery) e imprimimos na tela a mensagem.

Executando os códigos em Java

Vamos executar cada um dos códigos. Inicialmente, você deve se certificar de que o servidor RabbitMQ está executando. Em seguida, executamos o Produtor.java. Ele não produz nenhuma mensagem, mas envia a mensagem “Ola Mundo!” para o servidor.

Em seguida, executamos o Consumidor.java e obtemos:

plain-text

Mensagem recebida: Ola Mundo!

Desenvolvendo modelo publish/subscribe em Python

Confira o desenvolvimento do modelo publish/subscribe em Python!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Agora que desenvolvemos dois exemplos simples de consumidor/produtor para uma fila (Queue) em Python e Java, vamos avançar um pouco mais e ver um modelo publish/subscribe em Python. Assim, para que os exemplos de código funcionem, será preciso instalar a biblioteca pika do Python.

No modelo que desenvolveremos, vamos entregar uma mensagem para múltiplos consumidores, que é o padrão publish/subscribe. Para isso, construiremos um sistema de registros de eventos (logs) que consiste em dois programas:

O primeiro

Que emitirá mensagens de registro.



O segundo

Que receberá as mensagens de registro e imprimirá na tela.

Nesse sistema, qualquer “cópia” do receptor é capaz de consumir as mensagens. Dessa forma, poderemos executar duas cópias do nosso receptor (ou consumidor): uma enviará os registros para a tela e o outro para o sistema de arquivos.

Vamos além! dessa vez utilizaremos melhor a exchange. Para isso, vamos rever alguns conceitos:

- Um produtor é uma aplicação que envia mensagens.
- Um receptor ou consumidor é uma aplicação que recebe mensagens.
- Uma fila é um “buffer” que armazena mensagens.

A ideia central no modelo de mensagens do RabbitMQ é que o produtor **nunca** envia mensagens diretamente para uma fila. Na verdade, muitas vezes o produtor nem sabe se uma mensagem será entregue em alguma fila.

Exchanges

Na implementação do RabbitMQ, o produtor só pode enviar mensagens para uma Exchange, que recebe as mensagens e encaminha para as filas. A exchange deve saber exatamente o que fazer com uma mensagem que recebe. O tipo de exchange que define se ela encaminhará a mensagem para uma fila específica ou a muitas filas, ou mesmo será descartada. Existem poucos tipos de exchanges disponíveis:

- direct
- topic
- headers
- fanout

Aqui utilizaremos apenas a exchange fanout, que é muito simples. Ela apenas transmite todas as mensagens que recebe para todas as filas que conhece. É exatamente isso que precisamos.

O código em Python para definir uma exchange como fanout é o seguinte:

```
python
channel.exchange_declare(exchange='logs',
                        exchange_type='fanout')
```

Executando

Para publicar nesse exchange, faremos:

```
python
channel.basic_publish(exchange='logs',
                    routing_key='',
                    body=message)
```

Filas temporárias

Estávamos usando filas que tinham nomes específicos, como a “hello”. Dar um nome a uma fila é importante quando você deseja compartilhar a fila entre produtores e consumidores. Mas esse não é o caso no nosso sistema de registro. Queremos ouvir sobre todas as mensagens de log, não apenas um subconjunto delas.

Também estamos interessados apenas nas mensagens que circulam atualmente, e não nas antigas. Para resolver isso, precisamos de duas coisas:

Primeira

Sempre que nos conectamos ao servidor do RabbitMQ, precisamos de uma fila nova e vazia. Para fazer isso, poderíamos criar uma fila com um nome aleatório ou deixar o servidor escolher um nome de fila aleatório para nós, atribuindo o valor “” para queue_declare. Assim temos:

```
python
result = channel.queue_declare(queue='')
```

Nesse ponto, result.method.queue contém um nome de fila aleatório. Por exemplo, pode parecer amq.gen-JzTY20BRgKO-HjmUJjOwLg.

Segunda

Uma vez que a conexão do consumidor é fechada, a fila deve ser excluída. Para isso, utilizamos exclusive=True. Sendo assim, temos:

```
python
result = channel.queue_declare(queue='', exclusive=True)
```

Código em Python

A seguir, temos o código do produtor de mensagens (**produtor.py**).

```
python

#!/usr/bin/env Python
import pika
import sys

credentials = pika.PlainCredentials('admin', '123')
connection = pika.BlockingConnection(pika.ConnectionParameters('192.168.56.101',
5672, '/', credentials))

channel = connection.channel()

channel.exchange_declare(exchange='logs', exchange_type='fanout')

message = ' '.join(sys.argv[1:]) or "info: Hello World!"
channel.basic_publish(exchange='logs', routing_key='', body=message)
print(" [x] Sent %r" % message)
connection.close()
```

A única mudança em relação ao que já foi visto em códigos anteriores é a linha 10, que define a exchange como fanout.

A seguir, temos o código do consumidor de mensagens (consumidor.py).

```
python

#!/usr/bin/env Python
import pika

credentials = pika.PlainCredentials('admin', '123')
connection = pika.BlockingConnection(pika.ConnectionParameters('192.168.56.101',
5672, '/', credentials))

channel = connection.channel()

channel.exchange_declare(exchange='logs', exchange_type='fanout')

result = channel.queue_declare(queue='', exclusive=True)
queue_name = result.method.queue

channel.queue_bind(exchange='logs', queue=queue_name)

print(' [*] Waiting for logs. To exit press CTRL+C')

def callback(ch, method, properties, body):
    print(" [x] %r" % body)

channel.basic_consume(queue_name, on_message_callback=callback, auto_ack=True)

channel.start_consuming()
```

Aqui, a diferença em relação ao que já foi visto em código anterior está nas linhas 9 e 11. Na linha 9, definimos a exchange como fanout, e na 11 utilizamos o `exclusive=True`, para que a fila seja deletada após o uso.

Executando

Para executar o código em Python, lembre-se de verificar se o servidor RabbitMQ está funcionando. Inicialmente, execute as instâncias do consumidor. A primeira enviará as mensagens para um arquivo (file.log) e a outra apresentará as mensagens na tela.

```
plain-text
```

```
$ Python consumidor.py > file.log  
$ Python consumidor.py
```

Finalmente, podemos executar o produtor.py.

```
plain-text
```

```
$ Python consumidor.py
```

Você deverá observar que a mensagem gerada pelo produtor.py é consumida por ambas as instâncias de consumidor.py.

Verificando o aprendizado

Questão 1

Com relação ao message broker RabbitMQ, assinale a alternativa verdadeira.

A

As mensagens enviadas para o RabbitMQ são recebidas do consumidor e colocadas diretamente na fila.

B

O papel da exchange no RabbitMQ é verificar se alguém consumiu uma mensagem e, caso contrário, enviar essa mensagem para dead letter queue.

C

A RabbitMQ permite a comunicação por diversos protocolos e os consumidores/produtores podem ser desenvolvidos em diversas tecnologias.

D

Os componentes (produtor/consumidor) em RabbitMQ devem ser desenvolvidos em Python.

E

Os componentes (produtor/consumidor) em RabbitMQ devem ser desenvolvidos em Java.



A alternativa C está correta.

Além de ser open-source, o RabbitMQ implementa diversos protocolos, como AMQP, STOMP, MQTT e HTTP para comunicação, permitindo que os componentes (consumidor ou produtor) possam ser desenvolvidos em diversas tecnologias, contanto que implementem esses protocolos.

Questão 2

Sobre as exchanges do RabbitMQ, assinale a alternativa verdadeira.

A

As exchanges verificam as credenciais do consumidor ou produtor.

B

Um produto de mensagens em Python não pode enviar uma mensagem para um consumidor em Java, pois a exchange em Python é diferente da exchange em Java.

C

Existe uma única exchange no RabbitMQ que é excluída e criada toda vez que uma nova mensagem é enviada para o servidor.

D

As queues recebem as mensagens e enviam para a exchange correta de forma que o consumidor possa recebê-la.

E

Para escolher a exchange default, em Python, utilizamos o valor da string vazia "".



A alternativa E está correta.

De fato, as exchanges são responsáveis por receber toda e qualquer mensagem de um produtor. É possível selecionar a exchange default para enviar as mensagens, que geralmente, é representada como uma string sem conteúdo em Python e Java.

Limitações do REST

Confira as principais limitações do REST!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Motivação

O GraphQL (**Graph Query Language**) foi introduzido pelo Facebook em 2012 (e lançado publicamente em 2015). A funcionalidade que o Facebook estava desenvolvendo era apresentar para o usuário uma lista de posts. Para cada post deveria vir uma lista de usuários que curtiram o post. Para cada usuário deveria vir um conjunto de informações como a foto de perfil, o link para o Facebook do usuário e se o usuário já era amigo de quem estava acessando a funcionalidade.



Comentário

Essa é a descrição da funcionalidade quando acessada via navegador. Mas se acessada pela aplicação de celular, as informações mais detalhadas de cada usuário não aparecem. Isso porque o celular geralmente usa a rede 3G ou 4G, que é mais limitada quanto à largura de banda.

Na programação convencional, teríamos duas implementações de funcionalidades bem similares:

A primeira

Retorna mais informações para quem usa computador.



A segunda

Retorna menos informações para quem acessa via aplicativo de celular.

Aqui entra o GraphQL: ao invés de criar várias APIs similares, durante a solicitação você enviará quais dados deseja e somente estas serão enviadas de volta. Tudo isso utilizando a **mesma API**.

Entendendo o problema

A forma como uma aplicação se comunica com a outra evoluiu no tempo. A linha do tempo a seguir apresenta a evolução da forma de comunicação entre sistemas.

1981

RPC (Remote Procedure Call).

1991

CORBA (The Common Object Request Broker Architecture).

1998

SOAP (Simple Object Access Protocol).

2000

REST (Representational State Transfer).

2012

GraphQL (Graph Query Language).

O que você precisa saber é que o REST (2000) ainda é a forma de comunicação entre sistemas mais utilizada para o desenvolvimento de APIs. O GraphQL foi uma abordagem desenvolvida pelo Facebook para suplantando certas barreiras do REST.

Entendendo o REST

REST é uma API de transferência de estado representacional (Representational State Transfer) e cada "chamada" é denominada endpoint. A seguir, temos um exemplo de uma **API REST** e sua descrição.

GET /user

Busca uma lista de todos os usuários.

GET /user/:id

Busca um único usuário :id.

GET /user/:id/projects

Busca todos os projetos de um usuário.

O REST é um conceito de arquitetura de software baseado em rede e que não possui um conjunto de ferramentas oficial ou nenhuma especificação, podendo utilizá-lo com o protocolo HTTP ou AMQP. A ideia por trás do REST é desacoplar uma API do cliente e, por sua simplicidade, ele foi adotado por grandes empresas, como o Twitter.

Agora, vamos compreender o problema do REST. Considere os cards anteriores nos quais há 3 endpoints e considere se precisássemos dos projetos e atividades relacionados a esse projeto de um usuário. Veja que o endpoint "**GET /user/:id/projects**" traz apenas as informações dos projetos de um usuário, isto é, ele retorna para o cliente apenas a lista de projetos para determinado usuário, mas não retorna as tarefas/atividades relacionadas aos projetos.

Para atender a essa necessidade, a equipe de desenvolvimento teria algumas alternativas:

GET /users:id/projects?include=tasks

Incluir uma consulta e depois retornar as tarefas para cada projeto.

GET /projects?userid=:id&include=tasks

Separar o endpoint considerando o projeto e pedir a um aplicativo cliente a filtragem de consulta com base no id do usuário.

GET /tasks?userid=:id

Incluir todos os dados possíveis associados ao usuário em um único endpoint.

Não importa qual vai ser a abordagem, um novo trabalho em uma nova funcionalidade no back-end que processe a solução escolhida (uma das 3 anteriores, por exemplo) é necessário.

Conceito de over-fetching e under-fetching

Esses são dois conceitos importantes quando desejamos entender melhor a aplicabilidade do GraphQL. **Over-fetching** ou excesso de dados ocorre quando o resultado de uma solicitação possui mais dados do que o necessário.



Exemplo

Você deseja a lista de atividades de determinado projeto e o endpoint (supondo que seja uma solução REST) retorna todas as atividades de todos os projetos.

Já o **under-fetching** ou dados insuficientes é quando o resultado de uma solicitação retorna apenas parte da informação necessária.



Exemplo

Nesse caso você precisa da lista de todos os projetos de todos os usuários, mas o endpoint retorna apenas o projeto de um único usuário. Então, é necessário fazer um programa que leia a lista de todos os usuários e itere um a um nesse endpoint para obter a lista de todos os projetos de todos os usuários.

Visão geral do GraphQL

Confira um resumo do GraphQL!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

GraphQL

GraphQL é uma linguagem de consulta (**query language**), especificação e coleção de ferramentas desenvolvida para operar sobre um único endpoint por meio do HTTP, que otimiza o desempenho e a flexibilidade, minimizando os problemas do REST.

Mas não se pode dizer que é melhor ou pior que o REST, depende da natureza do seu projeto. Basicamente, o REST é o padrão convencional para o desenvolvimento de APIs e o GraphQL é uma linguagem de consulta que ajuda a resolver problemas com as APIs.

Confira as diferenças!

GraphQL

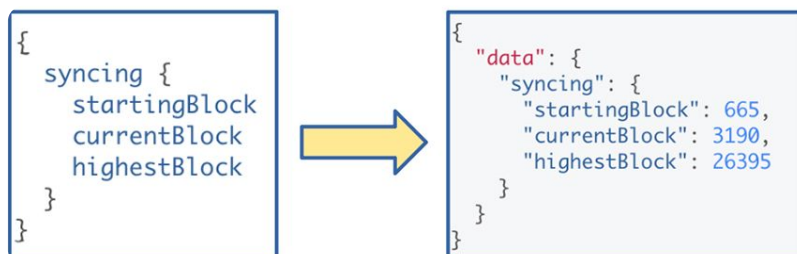
Linguagem voltada ao cliente. Sua arquitetura permite que o aplicativo front-end determine quais dados devem ser pesquisados e a quantidade de dados que deve ser retornada ao servidor.



REST

Define como as solicitações chegarão ao endpoint e o resultado que será retornado. Tudo é projetado no servidor.

Observe a seguir o exemplo de uma requisição GraphQL. O requisitante informa quais campos ele deseja e a resposta é exatamente o que é solicitado.



Requisição e resposta no GraphQL.

E aqui está a força do GraphQL: permitir definir exatamente o que é necessário para o cliente!

Observação: Na imagem anterior, o **Schema** que representa os dados poderia ter muitos outros campos além de startBlocking, currentBlocking e highestBlocking, tantos outros quanto forem necessários!

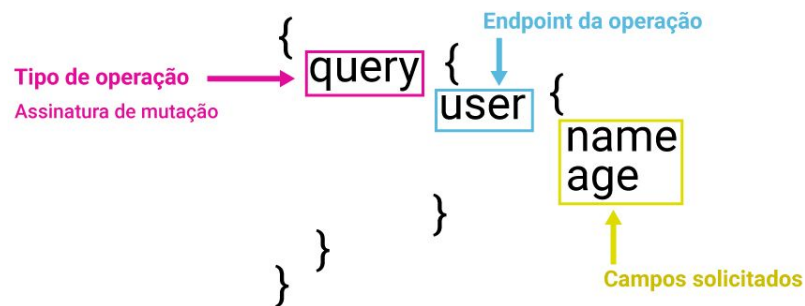
Motivos para usar o GraphQL

Devemos usar o GraphQL a fim de resolver os problemas com o REST, além de garantir algumas vantagens:

- Ideal para quando desejamos melhorar o desempenho de rede ao enviar menos dados ou somente informações necessárias e relevantes aos clientes.
- Ideal para que o cliente tenha controle dos dados que devem ser retornados.
- Ideal para atender a uma gama de clientes que precisam dos mesmos tipos de informação, mas de forma diferente. Por exemplo, você armazena as informações de usuários, projetos e atividades de projetos. Um cliente precisa da lista de todas as atividades por projeto e outro precisa de todas as atividades por usuário.

Estrutura geral e termos do GraphQL

Veja a forma como uma consulta (query) é estruturada em GraphQL. É possível identificar 3 partes principais:



Estrutura geral de uma solicitação GraphQL.

Existem diversas implementações de GraphQL em diversas linguagens de programação como JavaScript, Python, JAVA, Go, entre outras. Cada linguagem de programação tem sua própria implementação de GraphQL, mas você não precisa se preocupar com os detalhes de cada implementação. Entretanto, existe uma padronização de certos termos utilizados nas implementações.

É interessante que você conheça esses termos, pois eles ajudarão a compreender a forma de utilizar a implementação escolhida. Confira:

Types (tipos)

O modelo de dados é representado em types, que são fortemente tipados (typed). Parece redundante, mas não é! Deve existir um mapeamento one-to-one entre seus modelos e os códigos do GraphQL.

Queries (consultas)

As queries definem quais consultas você pode executar seu GraphQL. Por convenção, deve haver um RootQuery, que contém todas as consultas existentes.

Mutations (mudanças de conteúdo)

Se as consultas (queries) são solicitações GET, as mudanças de conteúdo podem ser vistas como solicitações POST | PATCH | PUT | DELETE que modificam os dados. Um exemplo de mutation de "getUser" (obter usuário) seria "createUser" (criar usuário), "updateUser" (atualizar usuário), "removeUser" (remover usuário).

Input

Tipo de objeto que agrupa um conjunto de parâmetros e pode ser utilizado como argumento em outro campo.

Resolvers

Função responsável por popular um único campo do seu schema.

Schema

Definido por meio de tipos, consultas e mudanças de conteúdo, sendo o que o endpoint do GraphQL expõe para os clientes.

Você pode ter ficado confuso com todos esses conceitos, mas eles ficarão mais claros daqui para frente!

GraphQL em Python

Confira como utilizar GraphQL em Python!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Graphene

Graphene é uma biblioteca que fornece ferramentas para implementar uma API GraphQL em Python usando uma abordagem code-first.

Mas o que é isso?

Para entender melhor, compare a abordagem de code-first do Graphene para criar uma API GraphQL com abordagens de first-schema, como Apollo Server (JavaScript) ou Ariadne (Python).

Na abordagem first-schema, primeiro se define qual o **schema** dos dados na forma de uma estrutura utilizando a linguagem de definição de schema (schema definition language). A título apenas de exemplo, uma definição de schema em Ariadne:

```
plain-text

from ariadne import gql, load_schema_from_path
schema = load_schema_from_path("schema.graphql")
schema = load_schema_from_path("schema")
schema = gql("""
    type Query {
        user: User
    }
    type User {
        id: ID
        username: String!
    }
""")
```

Na abordagem schema-first é exatamente isso: definir o schema de dados em primeiro lugar.

Já na abordagem code-first, primeiro escrevemos código Python para descrever os dados fornecidos pelo seu servidor.

Graphene é compatível com os principais frameworks Web para fins de integração. Além disso, o Graphene produz esquemas aderentes com a especificação GraphQL.

Construindo um exemplo com Graphene

Vamos inicialmente, criar um schema GraphQL para dizer “ola” e “adeus” em Graphene. A solicitação será uma query solicitando apenas um campo “ola” e especificando um valor para o atributo “nome”. Observe:

```
plain-text

{
  ola(nome: "amigo")
}
```

A partir dessa requisição, a resposta (Response) contendo apenas os dados solicitados (nesse caso, as informações relacionadas ao campo “adeus” não serão enviadas).

```
plain-text

{
  "data": {
    "ola": "Ola amigo!"
  }
}
```

Antes de iniciarmos qualquer código, precisamos instalar o Graphene. Nos exemplos a seguir, estamos utilizando a versão 3.6.10 do Python.

```
plain-text

$ pip install graphene
```

Vamos criar um Schema básico!

Mas, e aí? Vamos começar pelo schema? Por que o Graphene não é schema-first?

Simples, estamos definindo o schema na forma de código Python!

```
python

from graphene import ObjectType, String, Schema
class Query(ObjectType):
    ola = String(nome=String(default_value="estranho"))
    adeus = String()
    def resolve_ola(root, info, nome):
        return f'Ola {nome}!'

    def resolve_adeus(root, info):
        return 'Adeus!'

schema = Schema(query=Query)
```

Um Schema GraphQL descreve cada campo (Field) no modelo de dados disponibilizados pelo servidor usando tipos escalares, como String, int e Enum, e tipos compostos, como List e Object. Nosso Schema também pode definir qualquer quantidade de atributos (Argument) para nossos campos (Field). Essa é uma maneira simples para uma Query descrever exatamente os requisitos de dados para cada campo (Field).

No código definimos o modelo de dados com dois campos: **ola** e **adeus** (linhas 3 e 4). O campo (Field) “ola” é um String com um atributo (Argument), nome que também é String e com valor padrão “estranho”. O campo (Field) adeus é uma String.



Atenção

Para cada campo (Field) em nosso Schema, devemos escrever um método Resolver para buscar os dados solicitados pela Query do cliente usando o contexto atual (root, info) e os atributos (Argument) (nome).

Assim, o Resolver `resolver_ola` (linha 5) recebe como parâmetro `root`, `info` e `nome`, e retorna a “Ola” concatenado com o valor do atributo `nome` passado. Se não houver, ele utiliza “estranho” (linha 3).

O mesmo raciocínio pode ser aplicado ao campo `adeus`. No entanto, para o campo `adeus` não foram definidos atributos (Argument). Com essa visão geral do código em Python, podemos entender por que o `graphene` é considerado `code-first`: porque o `schema` e detalhes do `schema` são definidos programaticamente na classe (classe `Query`).

Se estivéssemos em uma biblioteca `schema-first`, primeiro definiríamos o `schema`. No nosso caso, e para título de comparação, o `schema` de nossos dados é o seguinte:

```
plain-text
type Query {
  ola(nome: String = “estranho”): String!
  adeus: String!
}
```

No nosso código em Python, estamos retornando as informações sem acessar nenhum tipo de fonte de dados como banco de dados, árvore de diretórios ou sistema de arquivos. Estamos apenas retornando informação estática (que se encontra no código).

Finalmente, desejamos agora realizar uma query. Para isso, vamos agora analisar o código completo, acrescentando as linhas 11 a 18 ao código apresentado anteriormente.

```
python
from graphene import ObjectType, String, Schema
class Query(ObjectType):
    ola = String(nome=String(default_value="estranho"))
    adeus = String()
    def resolve_ola(root, info, nome):
        return f'Ola {nome}!'

    def resolve_adeus(root, info):
        return 'Adeus!'

schema = Schema(query=Query)
query_string = '{ ola }'
result = schema.execute(query_string)
print(result.data['ola'])

query_with_argument = '{ ola(nome: "Aluno") }'
result = schema.execute(query_with_argument)
print(result.data['ola'])
```

Nas linhas 12 a 14, estamos realizando uma query, solicitando o campo “ola”. Em seguida chamamos o `schema.execute` passando a query. O resultado esperado é “Ola estranho!”. Observe que nesse caso não estamos passando nenhum atributo (Argument) na query (linha 12).

Nas linhas 16 a 18, estamos realizando uma query passando um atributo (Argument) como parâmetro. Da mesma forma, executamos a query por meio da chamada do `schema.execute` (linha 17). Finalmente, apresentamos o resultado na tela. Aqui, o resultado esperado é “Ola Aluno!”.

```
python
$ Python exemplo.py
Ola estranho!
Ola Aluno!
```

Verificando o aprendizado

Questão 1

Com relação ao GraphQL, assinale a alternativa verdadeira.

A

Over-fetching é quando o resultado de uma solicitação é insuficiente para o cliente.

B

Under-fetching é quando o servidor está livre para receber mais requisições do tipo GraphQL.

C

GraphQL utiliza o REST para realizar as requisições.

D

O GraphQL utiliza o protocolo HTTP para se comunicar.

E

A troca de mensagens realizada pelo GraphQL é feita usando XML.



A alternativa D está correta.

O GraphQL é uma linguagem que permite que, nas solicitações realizadas, seja enviado exatamente o que se deseja como resposta. Nesse sentido, a comunicação é, comumente, mas não exclusivamente, realizada por meio do protocolo HTTP.

Questão 2

Considerando os motivos para se utilizar o GraphQL, assinale a alternativa que descreve um cenário ideal para sua utilização.

A

Quando o consumidor e produtor foram desenvolvidos utilizando a mesma tecnologia.

B

Quando o cliente quer definir quais dados ele deseja de uma solicitação.

C

Quando o produto, por questões de desempenho, quer responder somente a determinadas requisições.

D

Quando as requisições são realizadas por meio de POST e GET.

E

Quando estamos limitados ao desenvolvimento de uma solução de comunicação na linguagem de programação Python.



A alternativa E está correta.

A ideia do GraphQL é exatamente permitir que cada cliente defina exatamente que dados precisa de uma solicitação sem que seja necessário criar diversos endpoint distintos para atender cada cliente. Dessa forma, ele resolve o problema principal do REST, que é exatamente o de responder com os dados definidos em uma interface, independentemente do cliente.

Considerações finais

Como vimos, as tecnologias disponíveis para resolver o problema de comunicação entre sistemas de software na integração de sistemas são as mais diversas. Muitas delas facilitam o trabalho do programador/desenvolvedor ao encapsular detalhes da complexidade da solução. Outras permitem que sistemas desenvolvidos com tecnologias diferentes se comuniquem entre si. Com isso, o trabalho de integração fica bastante facilitado, o que é imperativo com a crescente demanda pela integração de sistemas.

Podcast

Para encerrar, ouça sobre o uso dos serviços de mensageria como ferramenta para integração de sistemas, os recursos existentes, suas vantagens e desvantagens.



Conteúdo interativo

Acesse a versão digital para ouvir o áudio.

Explore +

O site da rabbitmq.com possui muito material sobre o RabbitMQ, inclusive tutoriais e diversas soluções, além de documentação completa sobre essa solução.

Referências

RABBITMQ. **Documentation**: Table of Contents. Consultado na Internet em: 07 dez. 2022.

RICHARDS, M.; MONSON-HAEFEL, R.; CHAPPELL, D. A. **Java message service**: creating distributed enterprise applications. O'Reilly Media, Inc., 2009.

ROY, G. M. **RabbitMQ in depth**. Simon and Schuster, 2017.

SHANNON, C. E. **The mathematical theory of communication** by CE Shannon (and recent contributions to the mathematical theory of communication), W. Weaver. Champaign, IL, USA: University of Illinois Press, 1949.